

AgentWorkShop: Closed-Loop Recipe Optimization by Governed Agent Teams over Heterogeneous Industrial Protocols

Anonymous Authors

Abstract—Industrial recipe optimization requires more than selecting a promising setpoint: an intervention must respect equipment limits, use the correct protocol, and remain traceable when its outcome is judged or reversed. AgentWorkShop links agent-team coordination to industrial supervision through member-node capabilities, shared production context, and a governed write path. Engineering limits, recipe windows, and approval requirements constrain execution; optimization records distinguish command acceptance, observed process response, agent judgment, and compensation. The evaluation uses real protocol transports with simulated plants and scripted teams, keeping language-model decisions outside the measured execution path. The integrated evaluation passes 75 checks, rejects 24 of 24 configured out-of-constraint probes across four writable stacks, and blocks none of four legal writes. A four-arm ablation attributes window-violation prevention to the recipe interlock: removing it admits six violations across three repetitions, whereas bypassing readback verification alone does not affect rejection. A fixed controller achieves 0.966–0.970 of a simulator-derived grid reference across three seeds. A recorded team mission reduces film thickness from 26.90 to 25.48 μm through four governed writes on three parameters, meeting a $25.0 \pm 0.7 \mu\text{m}$ objective. The results support a common execution layer that separates authorization, device acceptance, and process success. Physical validation and controlled agent comparisons remain necessary before claiming field reliability or autonomous optimization superiority.

Index Terms—agent teams, closed-loop optimization, heterogeneous protocols, human-in-the-loop, industrial informatics, large language models, recipe optimization, supervisory control.

I. INTRODUCTION

SUPERVISORY process adjustment seeks to keep product quality within its target band while production continues. On a film line, screw speed, line speed, and zone temperatures influence thickness; on a biaxially oriented line, draw ratio and stretch temperature couple several sections. Each adjustment must respect equipment limits and the active recipe window. The engineering problem is therefore not only choosing a setpoint, but linking its authority, operating context, and observed effect throughout an iterative search.

Language-model agents are being explored for industrial diagnosis, engineering assistance, and PLC program generation [1], [2], [3], [4], including interaction with simulated controllers and machine demonstrators [5], [6]. Such agents can propose and coordinate interventions, but protocol connectivity alone specifies neither which team member may write a parameter nor which limits and approval rules apply. A supervisory loop must also distinguish an authorized proposal, a completed write, and a beneficial process response.

AGENTWORKSHOP addresses this integration problem through explicit member-node bindings and shared line/product/recipe/run identifiers. Nodes expose engineering quantities rather than register addresses; bindings grant deployed members scoped tool authority; and hypothesis records link changes to subsequent observations and judgments. Fig. 1 contrasts this organization with application-specific integration. Shared context supports attribution but does not establish atomic execution, physical safety, or optimization success.

The loop operates at seconds-to-minutes supervisory timescales; fast regulation and plant protection remain outside it. Manual bindings require approval, while automatic bindings retain the applicable execution checks without that wait. Judgment and compensation are separate operations, preserving the distinction between a rejected proposal, a driver-accepted write (readback-validated where supported), an ineffective adjustment, and an attempted restoration.

The contributions are threefold:

- 1) A **node-native supervisory model** connecting heterogeneous acquisition and control nodes, deployed team members, and production runs through explicit capabilities and shared identifiers. Recipe windows constrain the search; they are not inferred safe operating regions (Secs. III and IV-A).
- 2) An **implemented governed loop** linking source-dependent admission, intervention records with optional hypotheses, agent-submitted judgments, separately executed compensation, and human approval, with explicit execution boundaries (Sec. IV).
- 3) A **layered, traceable evaluation** separating protocol checks, fixed-controller performance, and scripted team missions (Sec. V). The reported decision policies are deterministic and the plants simulated; these tests characterize platform paths, not LLM reasoning or controller superiority.

The contribution is an execution and evidence framework for supervisory search, not a new control law, learning method, or safety certification. Configuration reuse is evaluated only within the implemented drivers and scenarios.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based manufacturing control provides the foundation for linking team membership to industrial capabilities [7]. Industrial LLM applications span knowledge-graph construction

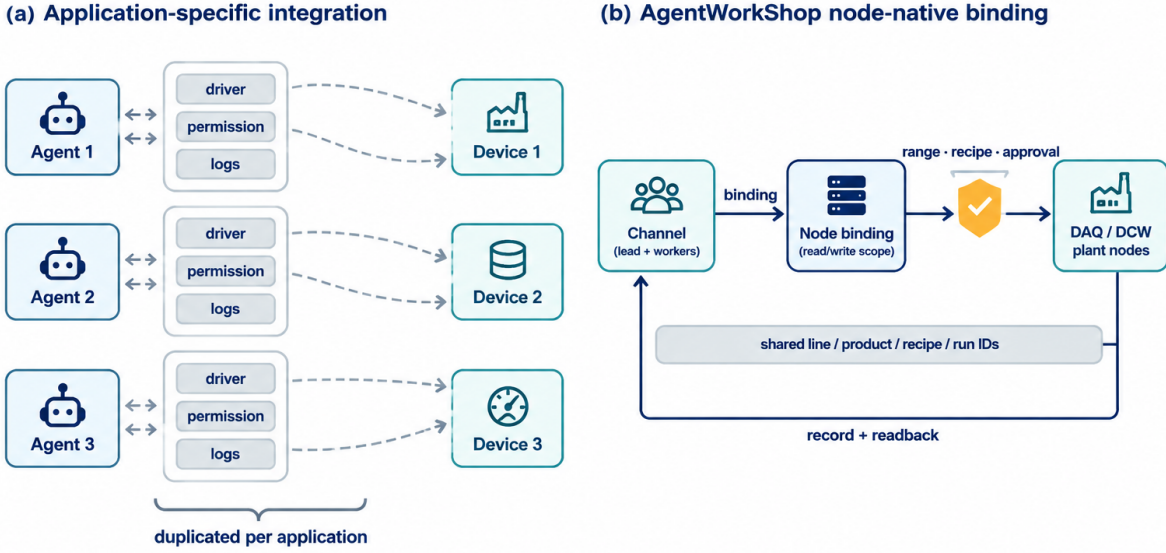


Fig. 1. From application-specific integration to explicit supervisory binding. (a) In advisory or application-specific integration, each agent reaches each device through its own integration, so driver, permission, and log handling are duplicated per application, and which member may move which parameter, under which recipe, is carried by the integration rather than by the platform. (b) The proposed organization: agent-team membership, explicit node capabilities, and execution feedback meet at shared production identifiers, so one iteration of the recipe search is comparable with the next; PLC regulation stays outside the agent loop. DAQ and DCW abbreviate the acquisition and data-control-write node types of Sec. III. Both panels are conceptual deployment patterns, not measured comparisons of competing systems or universal limitations of prior systems.

[8] and diagnosis [9], PLC program generation and verification [3], [4], and digital-twin configuration [10]. Applications that issue control commands include natural-language machine control over OPC UA [6] and LLM agents for simulated fault-tolerant control and twin-supported fault handling [5], [11]; these studies already validate corrective actions using process knowledge, deterministic checks, or twin simulation, so execution checks alone are not our novelty. InstructMPC injects language-derived context into model-predictive control [12]. Closest to our setting, agent-based supervision has been studied for service-oriented industrial cyber-physical systems [13], and LLM-based multi-agent systems coordinate shopfloor manufacturing tasks [14]; our emphasis is the explicit connection between deployed-member authority, source-dependent execution checks, and line/product/recipe/run attribution.

B. Orchestration, Industrial Models, and Execution Governance

AutoGen, LangGraph, and CrewAI organize agent tasks and tool use [15], [16], [17], and Model Context Protocol (MCP) and Agent-to-Agent (A2A) standardize tool and inter-agent interfaces [18], [19]; industrial integration additionally requires node semantics, driver behavior, and operating context. Our line/product/recipe/run organization is informed by ISA-95 and ISA-88 [20], [21]: line and product correspond to level-3 production work units and product definitions, run to production-performance records, and recipe to the ISA-88 master-recipe idea with grade and operating-point semantics; we do not implement the complete procedural models of either standard. Semantic asset models and gateway digital twins abstract heterogeneous equipment [22], [23], [24]; our focus is write governance over that abstraction, not a new twin model.

The ingredients of a governed write—node-scoped permission, range validation, approval, and an audit trail—have counterparts in mature DCS/SCADA/MES practice, typically enforced as role- and application-level controls under IEC 62443-3-3 [25]. LLM-mediated writes additionally require member-instance-level binding, semantic linkage between a write and the verdict or optimization record that produced it, and attribution across engine boundaries. Hard action shielding and control-barrier methods enforce formally specified admissible actions [26], [27]; run-time assurance and simplex-style architectures likewise separate supervisory logic from protected execution [28]. Our admission checks and rule-based backstop provide neither those formal guarantees nor certified plant protection. Human supervision also requires a clear division of responsibility, not merely an approval dialog [29], [30].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks evaluate agent behavior and policy compliance [31], [32]. Industrial datasets such as the Tennessee Eastman Process (TEP) and Secure Water Treatment (SWaT) support process-monitoring research [33], [34], but replay is not evidence of physical actuation, so we separate source checks, mock-driver probes, software-protocol integration, fixed-controller optimization, and exploratory LLM runs. Indirect prompt injection [35] and its benchmarked defenses [36] motivate checks on requests that compromised agents issue through legitimate tools; our evaluation includes no adversarial LLM probes.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP separates team coordination from plant interaction while linking both through industrial nodes and

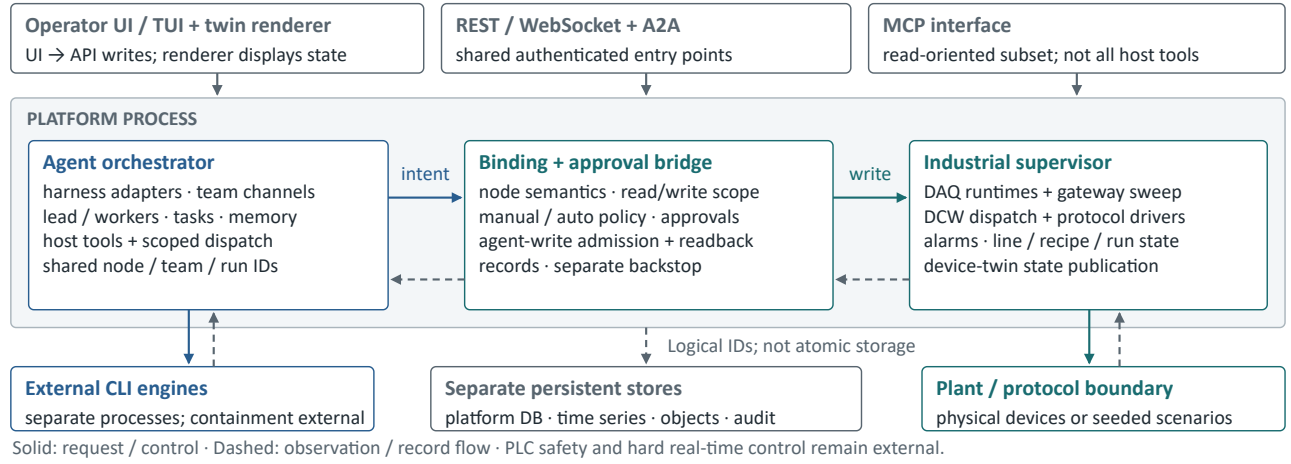


Fig. 2. System architecture and execution boundaries. The co-resident platform process hosts the agent orchestrator, the binding-and-approval bridge, and the industrial supervisor; operator entry points sit above it, and external CLI engines, persistent stores, and the plant boundary sit outside it. Shared identifiers support attribution across the logical data model; they do not make storage or physical actuation transactional.

production context (Fig. 2). A node is an acquisition or control endpoint with engineering semantics, connection settings, line membership, and agent bindings. Channels coordinate agents; controller services mediate node operations. The architecture covers the evaluated supervisory path.

A. A Shared Supervisory Core

The Node.js core hosts team routing, industrial services, and protocol adapters. Operator and integration interfaces share objects but differ in permissions; agent-tool authority comes from node bindings, not channel access or endpoint availability. Engine subprocesses and persistent stores remain external to the core. Connections, protocol queues, and readback are asynchronous; fast regulation and protection remain with the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

A channel coordinates deployed agent instances, called members, through a lead, workers, a task tree, messages, and retrievable memory. An agent proposes actions; its engine is the runtime implementation that executes it. Engine adapters preserve node identity and the controller interface across runtime implementations; the benchmark harness scripts these interactions for evaluation.

Acquisition nodes (DAQ) convert protocol readings to engineering quantities; control nodes (data-control-write, DCW) translate setpoints into device commands and readback results. Per-node runtimes manage timing and in-flight operations; controller services supply gateway state, recipe context, and write bookkeeping. Many-to-many node-device associations describe configuration, not execution routing: each control write uses the node's configured driver and connection.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs organize production context. Acquired data carry active-context tags resolved at ingestion. Optimization records retain line, node, recipe, and

actor references, but no run identifier; their baseline and response queries use node, line, and time rather than enforce run-isolated observations, so a baseline can overlap an earlier run. Platform state uses SQLite and file-backed repositories; configured measurement and object backends are TimescaleDB and MinIO, with SQLite and disk fallbacks. These links support attribution across stores, not a single transaction domain.

A BOPET thickness adjustment illustrates how the objects work together. The channel carries one production objective and delegates a task to a worker; the worker queries a thickness-acquisition node, then addresses casting speed, fast-roll speed, or rail width through its bound control nodes, each resolving its own protocol and engineering units while the line, recipe, and run identify the production context. An optimization record associates an executed change with its actor and later judgment, so the task can span protocols without treating a channel message as write authority or a task-board completion as a device acknowledgment; Sec. V-E follows this sequence in the recorded mission.

IV. CORE MECHANISMS

The governed loop connects semantic setpoints, member-level execution authority, and intervention records with optional hypotheses to judgment and compensation; the operator console exposes the same objects for commissioning and audit (Fig. 3), and Sec. V-A states the control structure that the evaluation exercises.

A. Optimization Objective and Search Space

The mechanisms exist to serve one task: move a product metric toward its target by choosing recipe parameters, and stop when further moves no longer help.

Decision variables. The recipe parameters bound to the line's control nodes—for a cast-film line, screw speed, line speed, and zone temperatures. A parameter is written in engineering units and resolved to a register only inside the

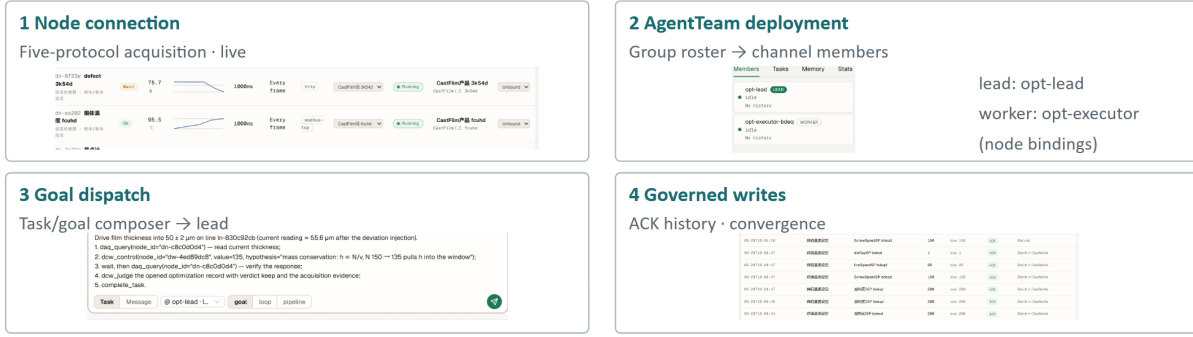


Fig. 3. The deployed AgentTeam workflow that exercises these mechanisms, shown as crops of the running console. (1) Commissioned nodes stream live acquisition over their protocols. (2) A group roster is deployed as a channel whose lead and worker carry the node bindings. (3) The goal composer hands the lead a thickness objective, which the lead decomposes into acquisition, governed write, verdict, and completion steps. (4) The parameter ledger records each governed write with its source and acknowledged value. The panels are interface records of the deployed workflow, not experimental measurements.

controller (Sec. IV-C), so a proposal carries physical meaning rather than an address. A commissioned node defines the quantity, units, calibration, limits, and line membership; protocol differences remain inside its driver. Member–node bindings, not channel access or protocol connectivity, grant agent-tool authority; they name deployed member instances rather than templates, so redeployment does not transfer them.

Objective and feasible set. The objective is the measured product metric, read back through the acquisition nodes of the same line: film thickness against its target band, defect rate, melt pressure, and the throughput and energy the operating point implies. Process effects require observations rather than write acknowledgments; freshness is exposed, but observation-quality gating is not enforced. The admissible setpoint interval is the intersection of the node’s hard range, active recipe window, and applicable product or baseline limits (Sec. IV-C). This interval constrains proposals; satisfying it neither certifies process feasibility nor discovers a new recipe window.

Iteration and termination. A hypothesis record links a proposal and write to an observation window and a submitted judgment. The supervisory strategy may stop at its acceptance band, on a favorable judgment, or at its write budget; these are policy decisions, not proofs of convergence.

B. The Agent Channel and Operator Surfaces

Channels route tasks among deployed members and retain messages and rationale. Task completion neither grants authority nor proves device execution. Industrial tools resolve the invoking member’s bindings for writes, judgments, and rollback; channel access and retrieved memory are not authorization. The scripted agent-param-tools check tests binding denial on the parameter layer (Sec. V-B).

C. Node Control Execution and the Parameter Layer

The parameter layer resolves semantic keys and units to control nodes and optional baseline limits, with active-product and recipe limits resolved dynamically, so agent operations need no register address, data type, or driver configuration. The semantic card supplies the engineering context—quantity, unit, precision, range, current value, freshness, and available

recipe information—and its suggested step size is advisory, not an admission rule. Acquisition runtimes track due times and in-flight samples under a rotating gateway sweep, providing supervisory rather than hard real-time sampling.

For an ordinary agent setpoint request, controller admission intersects the node’s engineering range with applicable parameter-baseline, active-product, and active-recipe limits. These checks depend on the entry route: recipe dispatch supplies a run identifier and omits the recipe-window layer; hold heartbeats bypass controller admission but share runtime write exclusion. Agent-origin ownership and directional cooldown checks are not universal guards. Line start installs an active run before dispatching recipe parameters individually. Stop clears that context, prevents subsequent acquisition dispatch, closes up to 500 open optimization records, and requests acquisition-offline status; it does not cancel in-flight samples, stop an already-running recipe dispatch, or block all control writes. Run tags are resolved at ingestion, so this command is supervisory batch management, not a PLC emergency stop.

Calibration maps engineering setpoints into the driver domain and available readback into physical units. Modbus and OPC UA drivers check readback using

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{rb} - v| \leq \tau_n, \quad (1)$$

where $[l_n, u_n]$ is the node range and d_n its display precision. For a nonzero linear calibration scale a , the driver receives $\tau_n/|a|$. Driver success is protocol-dependent: HTTP accepts a successful response without tolerance-checking an optional returned value, whereas MQTT publication supplies no readback. Only a validated readback supports setpoint agreement; neither that agreement nor request acceptance establishes process success.

Agent setpoint tools resolve bindings and obtain any required approval before controller admission, rechecking binding existence after approval. Admission checks pause and disable state, applicable ownership and directional cooldown, and the limits above; the runtime then handles write exclusion, connection, write, and readback. REST, recipe, and recovery routes have distinct entry authorization. Asynchronous connection and protocol queues prevent check–act atomicity against recipe changes, and write bookkeeping can fail without

reversing device execution, so attribution is not an atomic commitment across storage and the device. A stopped batch is not a physical safety barrier: the run gate limits acquisition, while plant-side protection remains independent.

D. Agent Closed-Loop Control

An intervention connects observation, proposal, execution, judgment, and possible compensation. The agent may attach an expected effect to its request, but the hypothesis field is optional. After driver-reported success, bookkeeping attempts to retain a change anchor. Agent- and rollback-origin writes can additionally open an optimization record with asynchronous baseline statistics; manual and recipe writes do not. The evidence associates a change with later observations; it is neither a causal experiment nor a guarantee that every accepted write was captured.

The five-second same-target rule suppresses bookkeeping, not the physical command: the driver has already executed when the latest node anchor is checked. Otherwise, the node's previous open record is closed before unchanged-value suppression; an unchanged command can therefore close a record without replacing it. Ownership is node-scoped: an agent-origin write can be blocked by another agent's non-stale open record. Directional cooldown restricts immediate movement away from a recently restored value; it does not exclude all manual, recipe, or system interventions. Defaults are a 600-s baseline lookback, a 300-s cooldown, and a 30-min stale-record interval. A missing prior value or a bookkeeping failure can leave recovery or attribution incomplete even when the driver accepted the request.

The judgment tool records the agent's reading of that comparison; it does not establish process success. An invocation of `dcw_judge` supplies a record identifier, verdict, and reason. A *keep* closes the record; for non-takeover judgments it also attempts to mark the currently active run as good, which is bookkeeping status rather than a certified snapshot of the operating point. *Rollback* and *uncertain* leave the record open. Setpoint restoration requires a separate rollback operation, and the strategy, not this status alone, decides whether to stop.

While the control gateway is running, its sweep schedules asynchronous checks of open records whose stored policy is not observe-only, by default no earlier than 120 s after the record is set and no more often than every 30 s. The manager prefers acquisition streams sharing the control node's primary-device alias, falling back to the line. It queries from the record's set time to now and sums out-of-window returned points or bucket averages across channels; a total of at least $B = 3$ triggers a rollback judgment. These need not be consecutive points, and B counts neither evaluator invocations nor raw sensor samples. Automatic recovery is attempted only while fewer than $K = 2$ retained records for the node have status *rolled-back*; the count spans retained history, not the current run, and does not bound every rollback route. At the threshold, or under an approval-required policy, the back-stop records escalation rather than writing. Query aggregation, asynchronous execution, and later record-state changes prevent these rules from establishing a worst-case recovery deadline or cancelling already-pending recovery.

E. Recipe Management and Physical Rollback

Recipe history preserves intended operation, not achieved device state. Versioned edits and known-good snapshots retain parameter sets and provenance, but reverting a definition creates a version with restored metadata without restoring PLC values. Applying a recipe creates a run and dispatches parameters; starting a line also installs the active context, so definition, run metadata, and physical parameters need not agree transactionally. Physical rollback issues a new, rejectable write to a recorded baseline or anchor-derived target; the run-related route uses the previous value of the last retained pre-start changed anchor, which need not be the actual setpoint at run start. Across nodes, compensation can partially succeed and cannot erase intermediate process states, so restored metadata, successful compensation, and verified process recovery remain distinct.

F. Human-in-the-Loop Control

Manual bindings add a timed approval wait to agent setpoint and rollback requests; auto bindings omit the wait but retain applicable execution checks. Reads and verdict recording do not require this approval. Setpoint requests show the member, node, requested value, request-time effective interval, and remaining time; rollback requests describe the target record or anchor. Approval reserves neither limits nor device state: binding existence and controller admission are checked again before execution. Expiry prevents execution and is recorded separately from refusal; duplicate pending requests for the same agent and node are suppressed, and unbinding can cancel outstanding requests. Tool approval differs from an optimization record awaiting human action: escalation does not execute recovery, and neither path covers an absent operator when $K = 2$ is reached. External engines still need deployment-level privilege and network restrictions: bindings govern sanctioned tool calls, not arbitrary I/O by a compromised process.

V. EXPERIMENTAL EVALUATION

A. Testbed, Scope, and Evaluation Flow

The experiments ask whether the platform (Q1) preserves write constraints across heterogeneous protocols, (Q2) separates prevention from readback and attribution, and (Q3) closes a team-level objective through traceable, multi-node interventions. We freeze the integrated archive $B = 20260920094610\text{-t04}$ from the September 20, 2026 consolidated report, recording seed 42, commit 1585948, harness hash `b203077a581aaba1` over nine benchmark-harness files, and Node `v24.19.0` on Windows `x64`. The command is `node bench/pipeline.mjs --profile integrated --seed 42 --cl-seeds 3`. Its immediately preceding same-version run is reported separately rather than selected for its endpoint, and the ablation and protocol/fault drill come from separate archives, not pooled with B . Numerical controller and mission observations use separate REST history queries that request one-second buckets and average up to six and five latest returned bucket means respectively; the archive preserves aggregates, not raw buckets.

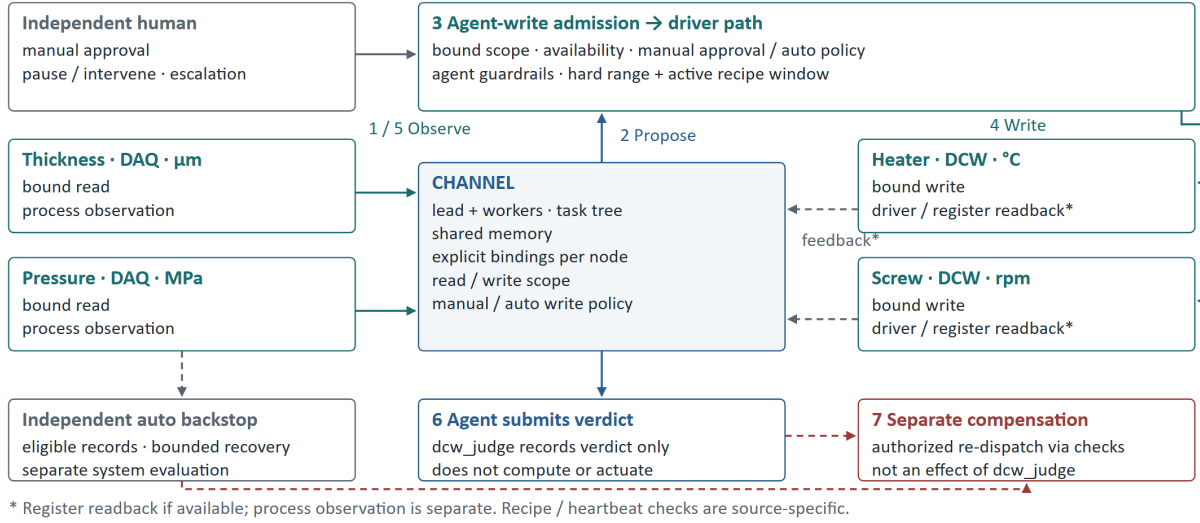


Fig. 4. The agent-team loop that the evaluation exercises, from the observation that starts an iteration to the compensation that can end one. The channel coordinates a lead and workers whose per-node bindings gate admission—bound scope, availability, approval policy, hard range plus the active recipe window—before a driver write with register readback where the protocol supports it. `dcw_judge` records the agent’s verdict but neither computes it nor actuates; compensation is a separate, rejectable write; the human authority and the auto backstop sit outside the channel. Numbered items are loop steps, not the subsections of Sec. IV; the plant is simulated.

The testbed runs actual Modbus TCP, OPC UA, MQTT, HTTP, and Modbus RTU-over-TCP transports against simulated plants, including a cast-film physics model, a strategy-based film line, and a biaxially oriented polyethylene terephthalate (BOPET) line. The integrated tool harness is deterministic `opcode`; mission leads and workers are scripted mock-engine members, and LLM decisions are disabled. Thus, the experiments evaluate supervisory execution and its evidence trail, not learned optimization, physical production performance, or an advantage over another agent framework. The simulator-derived reference and author-defined probes are internal oracles, not independent industrial benchmarks.

Because the loop is the object under test, Fig. 4 states what an iteration consists of before any number is reported: which signals return from the plant, where admission is decided, which node executes a write over which transport, and what evidence survives the write. The measured results then attach to those stages in a fixed order — coverage and protocol behavior (Sec. V-B), prevention versus readback (Sec. V-C), fixed-controller loop closure (Sec. V-D), and the recorded team-level mission (Sec. V-E) — so that a later claim can be traced back to the stage that produced it. Each subsection names its archive and states what the run does *not* establish, so the section reads as a sequence of independent checks rather than one aggregate score.

B. Integrated Coverage and Protocol Results

Archive *B* reports 75 passes, zero warnings, and zero failures across 17 phases. A pass is a check verdict, not an independent trial or a statistical confidence statement. The checks partition by function as 9 plant and provisioning, 5 protocol I/O and write governance, 7 three-cycle tool loops, 12 records, approval, recipes, and parameters, 16 team objectives and the BOPET line, 22 controller, scenario reuse,

TABLE I
PROTOCOL MEASUREMENTS IN *B*. LATENCY QUANTILES USE SIX TIMED WRITES PER STACK: P50 IS THE FOURTH SORTED VALUE AND P95 THE MAXIMUM. READBACK IS ONE SAMPLED DEVIATION, NOT AN ACCURACY BOUND. A DASH MEANS UNMEASURED OR INAPPLICABLE.

Stack	Samples	p50/p95 (ms)	Readback
Modbus TCP	9	49.232/101.164	0.04 °C
OPC UA	11	19.437/22.110	0 rpm
MQTT	12	19.427/23.084	—
HTTP	13	32.941/33.546	—
RTU over TCP	14	—	—

and backstop, and 4 multimodal acquisition and services. Acquisition-only nodes have no write endpoint; three omitted write/governance sub-checks must not be counted as tested writes. The optional LLM phase is absent. A separate source audit (20260920095700-1e3s) passes three checks, including 20/20 code-constant anchors, and the accompanying `_r7-apilive.log` records 60 API passes and two persistence-related skips despite the consolidated report’s zero-skip summary; these overlapping layers are not summed as independent experiments.

Five acquisition nodes store 59 samples. Four writable stacks each reject six out-of-constraint probes, giving 24/24 rejections and zero false blocks on four legal writes; the checker counts HTTP error responses to configured hard-range and window probes, so it does not verify each physical non-effect or establish adversarial robustness. Reprovisioning the second film-line scenario requires no driver-code changes: its three writable lines reject 9/9 probes without false blocks, alongside two acquisition-only satellites.

Table I retains the per-protocol measurements instead of reducing integration quality to one score. The report’s 30.259-ms aggregate is the arithmetic mean of the four protocol p50 values, not the median of pooled writes. MQTT and

TABLE II
FOUR-ARM GOVERNANCE ABLATION (THREE REPETITIONS, 18
OUT-OF-CONSTRAINT PROBES PER ARM). “BOUNDARY” COUNTS
DECISIONS MATCHING THE ARM-SPECIFIC BOUNDARY ORACLE;
“BREACHES” COUNTS EXECUTED WINDOW VIOLATIONS. THE
ENGINEERING GUARD REMAINS ACTIVE IN EVERY ARM.

Arm	Rejected	Breaches	False blocks	Boundary
Full	18/18	0	0	9/9
No interlock	12/18	6	0	9/9
No readback	18/18	0	0	9/9
Both disabled	12/18	6	0	9/9

HTTP archive no measured register readback, so their transport acceptance must not be described as verified actuator state. The approval request-to-decision latency is 448 ms with a scripted approver; human response time is not measured. The backstop rollback verdict is first observed at 130.205 s, and baseline restoration is verified subsequently, after an additional 1.5-s wait and read; the reported interval is therefore verdict-observation latency, not measured restoration time or a hard recovery deadline.

C. Ablating Prevention and Readback

The separate four-arm archive 20260920094442-elite uses seed 42 and three repetitions per arm. These are benchmark-only switches requiring `AW_BENCH_MODE=1`, not normal operator controls: the ablation bypasses soft-limit enforcement, readback-error enforcement, or both while retaining the engineering-range guard, and only the recipe soft layer is populated in this fixture. The report label “No readback” therefore means the tolerance is relaxed, not that readback I/O stops, and because the mock driver returns the submitted value without checking tolerance the switch is inert for mismatch detection here. The experiment isolates window rejection, not the effectiveness of readback verification. Table II shows all arms, including the executed violations: each repetition runs six out-of-constraint probes and evaluates three legal and three boundary cases separately, on new fixture objects from the same design. Arm-specific seeds vary numerical probes within the same case families, so these are not paired identical treatments or independent physical-plant trials. The arms keep the table order: 1 full, 2 no interlock, 3 no readback, and 4 both disabled.

Removing the interlock admits both window-class violations in each repetition, whereas bypassing readback verification does not change rejection (Table II). This supports a specific mechanistic claim: the window check prevents these violations; readback and journaling do not substitute for it. The fourth arm’s “Ungated” label is not guard-free: engineering limits remain enforced. All nine legal writes per arm are accepted. Boundary agreement means the expected accept/reject decision, not universal acceptance: full and no-readback reject six of nine boundary cases, whereas the two interlock-disabled arms accept all nine. Per-repetition p50 ranges are 110.6–141.0, 114.8–127.2, 111.9–127.1, and 124.8–139.9 ms in table order; each quantile uses 23 successful writes

(three legal and 20 timed), and arms run in fixed order. These descriptive ranges use mock in-memory readback, including in the verification-bypassed arms; they do not establish latency equivalence or the cost of removing physical readback I/O.

A second drill tests the same distinction on the live link. The archive 20260920094335-1b5g passes 13 checks (three source checks, six API checks, and four protocol checks); all five driver tests succeed. Its Modbus drill accepts a write in 25 ms, reads back a setpoint of 182, and observes the simulated process reach 179.5 after 30 s; five additional violations are rejected. After a frozen-process injection and subsequent write, the checker first observes a matching alarm at a rounded 2 s from its post-write timer, not 2 s from fault injection. A TCP disconnect/reconnect drill detects and recovers the connection. These narrowly exercised failures support the distinction between an accepted command and a responding process, not general fault tolerance.

D. Fixed-Controller Loop Closure

To isolate the execution path from agent strategy, the cast-film controller starts at screw speed $N = 150$ rpm, line speed $v = 95$ m/min, and zone setpoint 200°C . It maximizes

$$J = 55J_{\text{thick}} + 25J_{\text{quality}} + 8J_{\text{energy}} + 7J_{\text{throughput}}, \quad (2)$$

subject to melt temperature in $[195, 225]^\circ\text{C}$ and pressure at most 22 MPa. For thickness h in μm , defect percentage d in percentage points, and commanded speeds N and v in rpm and m/min, respectively,

$$\begin{aligned} J_{\text{thick}} &= \max\left\{0, 1 - \frac{\max\{|h - 50| - 2, 0\}}{10}\right\}, \\ J_{\text{quality}} &= 1 - \frac{\min\{d, 8\}}{8}, \\ J_{\text{energy}} &= 1 - \frac{N - 50}{150}, \quad J_{\text{throughput}} = \frac{v}{120}. \end{aligned} \quad (3)$$

On the configured nonnegative-defect and command domain, $J \in [0, 95]$; infeasible states are unscored. The denominator is the stored, three-decimal reference score, not a rescore of its separately rounded output variables. The fixed rule updates $N \leftarrow N(50/h)$ with a 22-rpm step limit, holds line speed fixed, and corrects heating outside $[196, 224]^\circ\text{C}$. The separately maintained simulator provides a finite-grid reference $J^* = 89.894$, not a proven global optimum: its grid ties the three zone temperatures and varies temperature from 195 to 225 $^\circ\text{C}$ in 2.5 $^\circ\text{C}$ steps, screw speed from 50 to 200 rpm in 5-rpm steps, and line speed from 20 to 120 m/min in 2.5-m/min steps, at a fixed 1.0-mm die gap. Unlike the online controller, the reference can vary line speed; the ratio therefore compares different action sets, not two equally equipped optimizers. Energy and throughput use issued setpoints rather than independent sensor measurements.

The reported mean ratio is 0.968 and mean wall time is 36.178 s per seed. Table III distinguishes the report’s endpoint statistic from the actual observations: the second write lowers J in every seed, so the trajectory is not monotone. Final thickness meets the reward band, and the residual gap to the reference lies in the quality, energy, and throughput terms

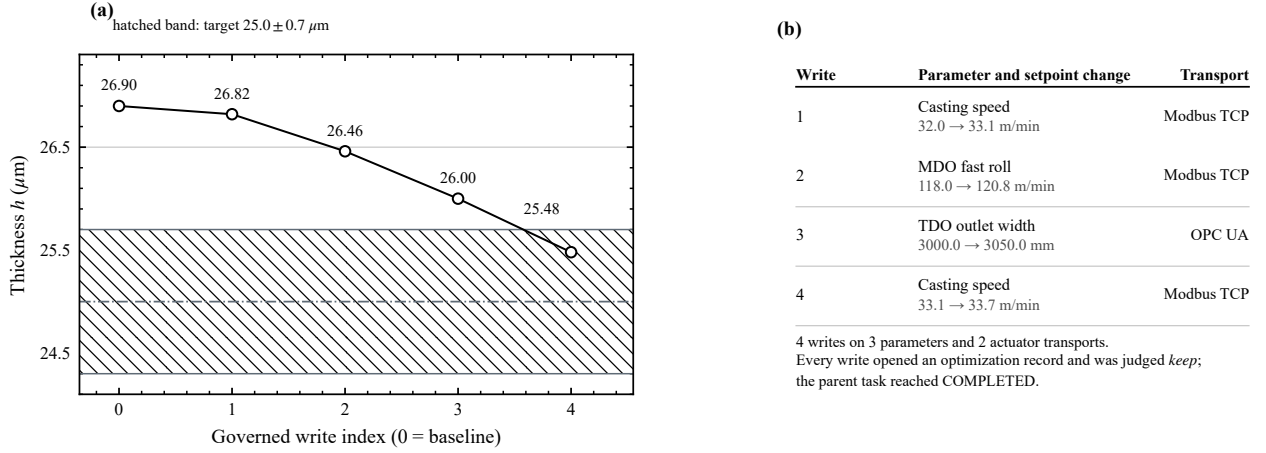


Fig. 5. Recorded scripted AgentTeam execution in *B*. (a) The five archived thickness observations against the $25.0 \pm 0.7 \mu\text{m}$ objective; markers are returned-bucket means from queries that requested one-second aggregation, and raw buckets and timestamps are not retained, so no continuous trajectory is inferred. (b) The four governed writes, each named by parameter, setpoint change, and transport. Every write opened an optimization record that received a logged *keep*, and the parent task reached COMPLETED. The plant is simulated and the policy is scripted, not an LLM-generated search.

TABLE III
FIXED-CONTROLLER RESULTS IN *B*. J_{end} IS THE MEAN OF THE TWO POST-WRITE EVALUATIONS, NOT THE LAST OBSERVATION. ALL SEEDS USE TWO ACCEPTED WRITES AND ZERO REJECTIONS.

Seed	J_0	J_{end}	J_{end}/J^*	Final h (μm)
42	70.167	86.845	0.966	49.200
43	68.608	87.236	0.970	49.683
44	65.740	87.080	0.969	50.100

rather than in thickness. Because the controller inverts the simulator’s own $h \propto N/v$ relation, this experiment shows loop closure under a known model, not general search efficiency or learned control superiority.

E. The Recorded AgentTeam Optimization Process

The first mission links task delegation, a five-minute acquisition query, a governed temperature write from 200 to 204.704 °C, a 204.700 °C observation, an attributed *keep* record, and parent-task completion. All six checks pass; the ± 0.75 °C tolerance makes this a workflow-closure test rather than a precision-control result.

The BOPET mission extends that workflow to nine simulated devices with 30 writable setpoints and 19 acquisition nodes, against a thickness objective of $25.0 \pm 0.7 \mu\text{m}$. Figure 5 reconstructs the actual intervention sequence from *B* rather than an illustrative optimization curve. Four of six allowed writes act on three distinct parameters: casting speed from 32 to 33.1 m/min, fast-roll speed from 118 to 120.8 m/min, outlet rail width from 3000 to 3050 mm, then casting speed again to approximately 33.7 m/min. The archived thickness observations are 26.90, 26.82, 26.46, 26.00, and 25.48 μm , so the endpoint sits 0.48 μm from the target and 0.22 μm inside the permitted band.

Each accepted write creates a distinct optimization record, followed by a settling gauge observation and a scripted *keep*

judgment. The policy uses relative-thickness-error gains of 0.5, 0.35, and 0.3 with fixed actuator rotation, so it does not discover the sequence. The mission logs no independent post-write register value for these actuators and the caller issues *keep* unconditionally, so the demonstrated contribution is scoped, attributable multi-node execution toward one objective rather than an independently validated judgment or an optimal recipe window. Melt-temperature observations remain between 291.1 and 291.525 °C, inside the configured 268–300 °C window; no disturbance tests that bound.

F. Repeatability and Limits of the Evidence

The immediately preceding same-seed, same-hash archive $A = 20260920093330\text{-md4}$ also passes 75/75 checks, rejects 24/24 primary and 9/9 second-scenario probes, accepts six controller writes, and closes the BOPET mission with four writes on three parameters. Their final thicknesses nevertheless differ, at 25.64 versus 25.48 μm , as do mean controller ratios (0.970 versus 0.968) and backstop verdict-observation latencies (130.182 versus 130.205 s). These are two consecutive regression runs, not a reliability campaign. The comparator reports matching verdicts, but its controller counter rows contain null fields and its protocol arrays are empty, so we check the underlying run records rather than treat its “REPRODUCIBLE” label as proof of full numerical agreement.

Earlier archives include process collapse despite a non-failing hard gate; those different-revision runs are not pooled with *A* and *B* or erased by their success. Initial simulator state, sampling timing, and shared-instance residue remain confounders, and the biax journal check has an empty evidence array although the mission log contains a casting-node journal excerpt. These limitations motivate pinned initial conditions, explicit missing-evidence failures, broader fault injection, and physical validation before deployment.

VI. DISCUSSION AND LIMITATIONS

A. What the Loop Delivers

Primary archive B is the reference for the results in Sec. V; earlier campaigns provide historical limitations, not the headline outcome. The evaluated paths connect engineering quantities, member-level authority, and production context to intervention and compensation records across implemented drivers. Their output is an executed setpoint and associated evidence within configured limits, not a validated safe operating interval.

The distinction is operationally useful because the remedies differ. A binding or range rejection calls for revising authority or the proposal; a transport or readback failure calls for inspecting execution; an accepted write followed by an out-of-band process response calls for a new strategy or compensation. The interlock ablation and frozen-process drill exercise two of these distinctions: attribution does not prevent an inadmissible command, and matching a setpoint does not establish a responding process. The platform makes these stages inspectable without claiming that the agent's judgment is an independent quality oracle.

Proposal, admission, readback, and process judgment remain distinct. `keep` records a submitted verdict; readback checks device-level agreement; sustained convergence requires separate process evidence. Observation-quality gating is not enforced, so a judgment may use stale or unrepresentative telemetry. The deterministic controllers and scripted members test these execution paths, not LLM optimization quality. Neither successful writes nor a passing regression gate establishes reliable process control.

B. Reproducibility and Threats to Validity

Commands, seeds, source fingerprints, and per-check evidence support reruns and archive comparisons, but deterministic decision policies do not make the whole experiment deterministic. Simulator initial state, asynchronous execution, and sampling windows can change outcomes. A harness fingerprint does not pin the separately versioned plant simulator, and working-tree hashes can depend on line endings. Exact reconstruction therefore requires the simulator version and reset state as well as the platform and harness sources.

The checks were developed against this implementation, and the reference J^* derives from the same simulator used by the controller. They form a regression suite rather than an external performance oracle. Historical warning-only mission outcomes show why an aggregate gate alone is insufficient, and the current archive also contains a journal check with an empty evidence array (Sec. V-F). Historical real-LLM campaigns used different scoring inputs and observation windows and remain exploratory, not comparable optimization evidence.

The study is simulation-tier: no physical-device validation, TEP or SWaT replay, human-operator study, or external-framework comparison is reported. Readback-consistent writes can still accompany adverse process behavior, and book-keeping can fail after actuation without reversing it. Device agreement, attribution, and process validity are thus separate properties.

C. Deployment Boundaries

Cross-agent oscillation within permitted ranges, long-horizon contention, storage-failure availability, and worst-case recovery time remain unevaluated. The backstop depends on data and policy; its wait and recheck cadence is not a recovery deadline. At its retained-history threshold $K = 2$, escalation replaces automatic restoration and may leave a record unresolved without an operator.

Bindings govern sanctioned tool calls, not arbitrary I/O by a privileged or compromised engine. Least privilege, endpoint security, and IT/OT network separation remain deployment responsibilities [37], [38]. Governed writes are not a certified safety instrumented function or an independent protection layer: controller limits, interlocks, and any required IEC 61511 protection must remain independent [39]. The platform neither detects nor compensates for their absence.

Manual approval can introduce fatigue and over-reliance [29], [30], [40], [41]. Role-, batch-phase-, and magnitude-specific policies and enforced segregation of duties remain extensions. Deployment also requires handling approval-channel and alarm-path failures [42], records left open by crashes, synchronized clocks, and off-host evidence.

VII. CONCLUSION

AGENTWORKSHOP connects deployed agent teams to industrial supervision through member-node capabilities, production context, and hypothesis-bearing intervention records. Source-dependent admission, submitted judgment, and separately executed compensation make the supervisory loop inspectable without making it atomic or safety-certified. Simulation-tier protocol checks, deterministic controllers, and scripted missions characterize implementation behavior, not LLM optimization or field reliability. Physical validation, stronger execution consistency, observation-quality gating, and controlled agent comparisons remain open.

DATA AVAILABILITY AND DISCLOSURE

The integrated archives identified in Sec. V retain per-check results in `run.json`, verdicts in `summary.json`, metrics in `metrics.csv`, and `agentteam-*.log` traces. The ablation supplies `run.json` and `el-lite.csv`; protocol and static suites supply `run.json` and `reports`. Figure scripts and archive-consistency checks accompany the project source. These records do not by themselves reconstruct the separately versioned simulator. Fig. 1 is an AI-generated illustration of two conceptual deployment patterns; Fig. 3 is a composite of crops from the deployed console; the remaining figures are programmatically rendered schematics or archive-derived plots. None presents an invented experimental trajectory. AI assistants supported manuscript editing, evidence checking, and figure preparation under author direction; the authors remain responsible for claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.

- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IGK: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, arXiv preprint arXiv:2505.02076.
- [12] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-llm-in-the-loop framework for context-aware control," *arXiv preprint arXiv:2504.05946*, 2025.
- [13] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [14] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [16] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [17] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [18] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024.
- [19] Linux Foundation, "Agent2agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025.
- [20] International Society of Automation, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [21] —, "ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1)," Standard, 2010.
- [22] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [23] International Electrotechnical Commission, "IEC 63278 (series): Asset administration shell for industrial applications," International Standard, 2023.
- [24] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, 2025.
- [25] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [26] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [27] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [28] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [29] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [30] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [31] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [32] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [33] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [34] J. Goh, S. Adep, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [35] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [36] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. USENIX Security Symposium*, 2024.
- [37] International Electrotechnical Commission, "IEC 62443: Security of industrial automation and control systems," International Standard, 2018.
- [38] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [39] International Electrotechnical Commission, "IEC 61511: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.
- [40] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [41] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [42] Engineering Equipment and Materials Users Association, "Eemua 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.